2600
2600, école de cybersécurité

MODULE
Noyau Linux

TP 03

Programmation du noyau Linux

Rootkit + Système de fichiers virtuel

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

Sommaire

Objetifs 3
LKM Rootkit : masquage de fichier avec ftrace 4
- Contexte 4
- Objectifs 4
- Contraintes techniques 4
- Indications 5
  - Structure ftrace_hook 5
  - Résolution du symbole 5
  - Flags obligatoires 5
  - Garde anti-récursion dans le callback 5
  - Parcours du buffer linux_dirent64 5
- Pour aller plus loin (optionnel) 5

LKM VFS `chiffrefs` 6

Objetifs

L'objectif de cet atelier est de réaliser des kernels modules offensifs:

- Un début de `rootkit`
- Un premier VFS

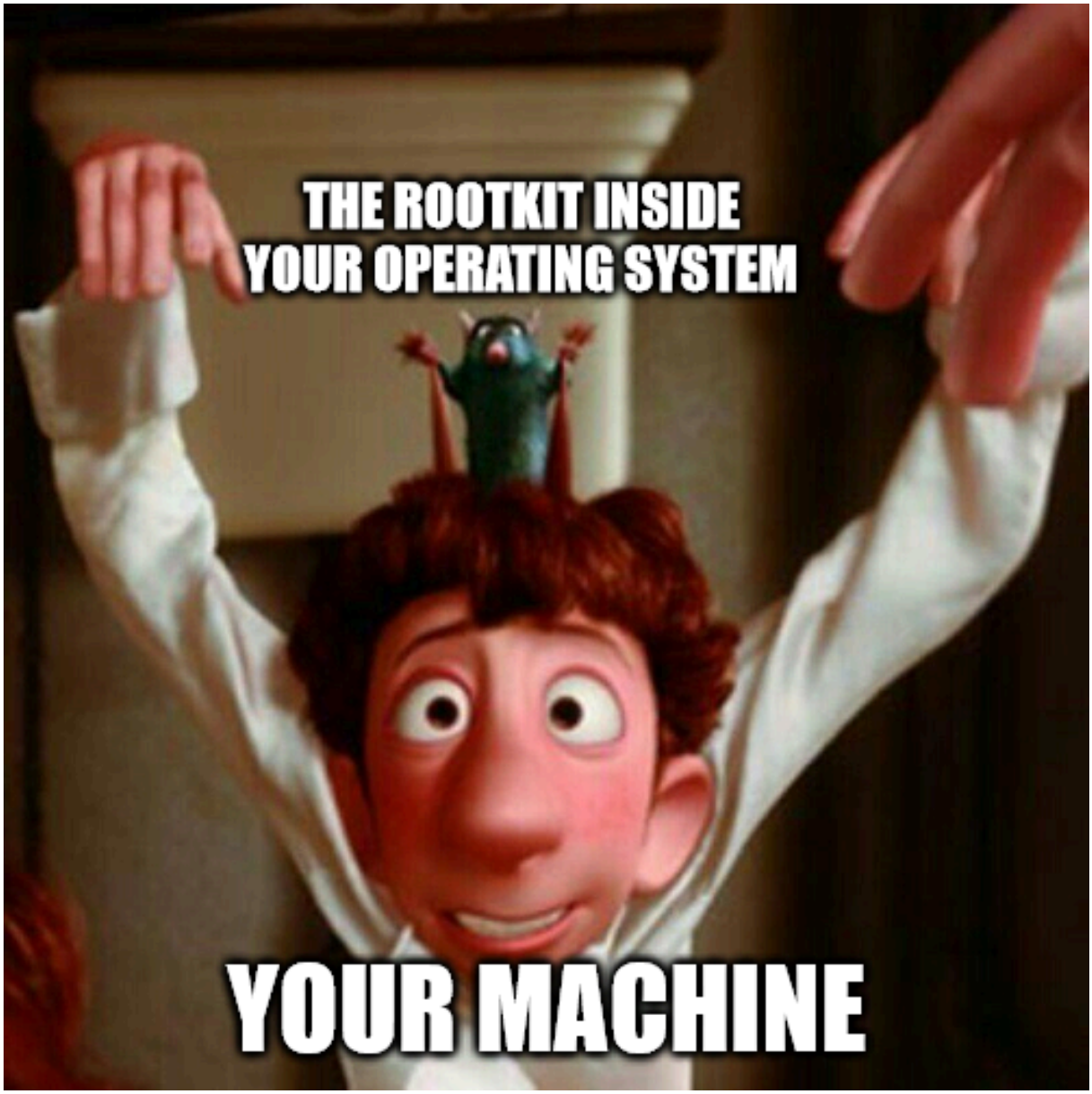


Fig. 1. – width:400px

LKM Rootkit : masquage de fichier avec ftrace

L'objectif de ce projet est de créer un module noyau qui utilise **ftrace** pour hooker l'appel système `getdents64` et masquer un fichier d'un listage de répertoire.

Contexte

`getdents64(2)` est le syscall utilisé par `ls`, `find` et `opendir()` pour lister le contenu d'un répertoire. En interceptant ses valeurs de retour, il est possible de **faire disparaître** une entrée aux yeux de l'espace utilisateur, sans toucher au système de fichiers réel.

Vous implémenterez ce mécanisme pour masquer le nœud `/dev/` créé par votre propre module (typiquement `/dev/rootkit` si vous enregistrez un `miscdevice`).

Objectifs

Vous devez écrire un module noyau `hidemod.ko` qui :

1. Enregistre un `miscdevice` nommé `rootkit` (visible initialement dans `/dev/`).
2. Installe un hook ftrace sur `sys_getdents64`.
3. Filtre les entrées dont le nom correspond à `rootkit` dans tout listage de répertoire (notamment `ls /dev/`).
4. Retire proprement le hook dans le `module_exit`.

Vérification attendue :

```shell
$ insmod hidemod.ko
$ ls /dev/rootkit          # doit exister réellement
/dev/rootkit
$ ls /dev/ | grep rootkit  # ne doit rien afficher
$                          # entrée masquée
$ rmmod hidemod
$ ls /dev/ | grep rootkit  # réapparaît
rootkit
```

Contraintes techniques

- Utiliser `ftrace` (pas de modification directe de `sys_call_table`).
- Ne pas patcher le bit WP du registre CR0.
- Copier le buffer `getdents64` en espace noyau (`kzalloc` / `copy_from_user`) avant de le filtrer, puis le renvoyer avec `copy_to_user`.

- Le `memmove` de suppression d'entrée doit correctement mettre à jour la valeur de retour (`ret -= cur->d_reclen`).

Indications

Structure ftrace_hook

Définir une structure `ftrace_hook` portant : le nom du symbole, le pointeur vers la fonction de remplacement, un pointeur vers lequel l'adresse originale sera écrite, et un `struct ftrace_ops`.

Résolution du symbole

Utiliser `kallsyms_lookup_name` (récupéré via `kprobe` comme vu en cours) pour obtenir l'adresse de `sys_getdents64`.

Flags obligatoires

```c
hook->ops.flags = FTRACE_OPS_FL_SAVE_REGS | FTRACE_OPS_FL_IPMODIFY;
```

Garde anti-récursion dans le callback

```c
if (!within_module(parent_ip, THIS_MODULE))
    regs->ip = (unsigned long)hook->function;
```

Parcours du buffer linux_dirent64

Les entrées sont contiguës. Avancer avec `bpos += cur->d_reclen`. Supprimer une entrée avec `memmove` puis `ret -= cur->d_reclen` sans incrémenter `bpos`.

Pour aller plus loin (optionnel)

- Paramétrer le nom à masquer via un paramètre de module (`module_param`).
- Masquer également le module lui-même de `lsmod` / `/proc/modules` :

```c
list_del_init(&THIS_MODULE->list);
```

- Hooker aussi `sys_getdents` (version 32 bits) pour une couverture complète.

Vous apprendrez :

- Comment utiliser ftrace pour hooker une fonction noyau.
- Comment parcourir et modifier en place un buffer `linux_dirent64`.
- Comment enregistrer et désenregistrer proprement un hook dans un LKM.

LKM VFS `chiffrefs`

L'objectif de ce projet est de créer un nouveau système de fichiers pour le noyau Linux.

Vous devez :

- Créer un nouveau système de fichiers.

Votre système de fichiers doit fonctionner comme une couche superposée (*overlay*) dans laquelle chaque octet écrit dans un fichier est décalé d'un nombre donné dans la chaîne d'options passée au montage sur le répertoire utilisé en arrière plan lors du montage.

Utilisation :

```shell
$ insmod chiffrefs.ko
$ mkdir chiffred clear
$ mount chiffred clear -t chiffrefs -o n=13
$ echo -n foobar > clear/test
$ cat clear/test
foobar
$ cat chiffred/test
sbbone
```

Vous apprendrez :

- Comment créer un nouveau système de fichiers.
- Comment utiliser des fichiers dans l'espace noyau.
- Comment construire un module Linux chargeable hors-arbre.
- Comment charger un module.
- Comment lire des paramètres de configuration du VFS lors d'un mount.
- Comment définir les opérations `open`, `read`, `write` etc... sur un VFS.